
Notes de référence pour l'entretien

Points clés et arguments à connaître pour l'oral

Document vivant : il s'enrichit au fil du projet, à chaque notion importante à retenir pour la présentation.

Table des matières

1	Pourquoi ne pas utiliser une IA de génération d'images	2
1.1	Le choix retenu	2
1.2	Pourquoi c'est le bon choix pour ce test	2
1.3	Le principe	2
1.4	Comment ajouter une vraie dimension « IA image » (pour aller plus loin)	2
1.5	En résumé	2
2	Un RAG aurait-il sa place dans ce projet ?	3
2.1	Le verdict en une phrase	3
2.2	Rappel : ce qu'est un RAG	3
2.3	Pourquoi ce n'est pas nécessaire pour le test	3
2.4	Où le RAG apporterait une vraie valeur (en V2)	3
2.5	La nuance d'ingénieur (à connaître)	3
2.6	Le lien avec la feuille de route de Gefradis	3
3	Comment l'application respecte toute la charte	4
3.1	Deux familles de règles, deux endroits	4
3.2	La charte visuelle : dans les gabarits et le code (déterministe)	4
3.3	La charte éditoriale : dans le prompt système	4
3.4	La garantie architecturale	4
3.5	En toute transparence : ce qui n'est pas encore en place	4
4	Résumer la charte dans le prompt fait-il perdre du contexte ?	5
5	Pourquoi Claude ne génère pas le HTML/CSS	5
5.1	Deux généricités à distinguer	5
5.2	Les risques si l'IA génère le HTML/CSS	5
5.3	La bonne façon d'être générique	6

1. Pourquoi ne pas utiliser une IA de génération d'images

1.1. Le choix retenu

L'outil n'utilise **pas** d'IA générative d'images (DALL-E, Midjourney, Stable Diffusion) pour créer les visuels. C'est le **code**, via Playwright, qui assemble des éléments graphiques *existants* (textes, couleurs, logo, gabarits). L'IA (Claude) intervient sur le **texte et la structure**, pas sur le dessin.

1.2. Pourquoi c'est le bon choix pour ce test

- **Respect de la marque (brand safety)** : une IA d'image est incapable de reproduire fidèlement une fenêtre ou une porte Gefradis précise. Elle invente des formes, et déformerait aussi le logo.
- **Le texte** : les modèles d'images écrivent encore mal le texte (fautes de frappe, lettres déformées). Or nos bannières sont essentiellement du texte.
- **Les dimensions exactes** : générer une image au format exact (320x50, 160x600...) avec une IA donne souvent des visuels écrasés ou rognés.

1.3. Le principe

On utilise l'IA là où elle excelle (la réflexion, le copywriting, la structure) et le code là où il est imbattable (la précision géométrique et le respect de la charte). **Claude pour le texte, Playwright pour l'assemblage.**

1.4. Comment ajouter une vraie dimension « IA image » (pour aller plus loin)

Deux approches possibles, sans casser l'architecture :

Option A : l'IA génère le fond d'ambiance. L'utilisateur demande un fond personnalisé via une API d'image (DALL-E 3, Stable Diffusion). Exemple de prompt envoyé par l'application : « Une maison moderne lumineuse avec de grandes baies vitrées, style épuré, flou d'arrière-plan, tons bleus et blancs sourds. » Le code récupère cette image, la place en arrière-plan, et Playwright superpose par-dessus le logo Gefradis parfait, les textes nets de Claude et le bouton d'action.

Option B : l'IA choisit le meilleur asset. Si Gefradis fournit une banque d'images (`fenetre_pvc.png`, `porte_aluminium.png`, `volet_roulant.png`), Claude analyse le brief et renvoie le bon fichier dans son JSON, par exemple `"asset_a_utiliser": "fenetre_pvc.png"`. Le code va alors chercher automatiquement le bon visuel. C'est une utilisation intelligente de la gestion des assets.

1.5. En résumé

Le projet contient bien une dimension IA : un LLM (Claude) pour l'adaptation intelligente des messages et la structure du contenu. La génération d'image pure est volontairement écartée pour garantir la fidélité au produit et au logo.

Phrase à dire à l'oral

« J'ai exclu la génération d'image pure par IA pour garantir que le produit Gefradis et le logo soient reproduits à 100% sans hallucination. En revanche, l'IA intervient intelligemment dans la conceptualisation textuelle et contextuelle de la bannière. Si besoin, on peut tout à fait connecter une API de génération d'image (par exemple DALL-E) pour concevoir uniquement les fonds d'ambiance. »

2. Un RAG aurait-il sa place dans ce projet ?

2.1. Le verdict en une phrase

Pas de RAG pour le rendu du test : le cœur de la valeur IA, c'est le **workflow LLM** (du brief vers le copy, puis vers la mise en page), pas la recherche documentaire. Le RAG devient pertinent **en V2**, quand la base de connaissances grandit.

2.2. Rappel : ce qu'est un RAG

Un RAG (*Retrieval-Augmented Generation*) va chercher, dans une base de documents, les passages utiles, puis les injecte dans le contexte du modèle au moment de répondre. Il sert quand la connaissance est **trop volumineuse** ou **trop changeante** pour tenir dans le prompt.

2.3. Pourquoi ce n'est pas nécessaire pour le test

- La charte actuelle (ton, couleurs, arguments) tient en quelques phrases : elle va directement dans le **prompt système**. Un RAG par-dessus serait de la sur-ingénierie.
- Le cas d'usage est direct : un brief en entrée, un kit en sortie. Le LLM seul suffit.
- Savoir *ne pas* ajouter un outil prématuré est aussi une compétence d'ingénieur.

2.4. Où le RAG apporterait une vraie valeur (en V2)

- **Sélection automatique des visuels** : avec une banque de photos produit (fenêtres, portes, volets) décrites par des métadonnées, retrouver le bon visuel pour un brief donné. C'est l'évolution naturelle de l'Option B.
- **Réutilisation des campagnes passées** : retrouver les opérations similaires déjà réalisées, pour garder une cohérence et inspirer le copy.
- **Charte et guidelines détaillées qui évoluent** : quand la documentation marketing devient volumineuse et change souvent, on la met à jour dans la base, sans toucher au code.

2.5. La nuance d'ingénieur (à connaître)

- **C'est une question d'échelle** : le RAG se justifie quand la base est grande et évolutive ; sur une petite charte fixe, le prompt suffit.
- **Retrouver un asset n'exige pas forcément un RAG complet** : pour une banque modeste, une simple correspondance par métadonnées, ou un choix par le LLM dans une liste, suffit. La base vectorielle (*embeddings*) se justifie sur un grand volume.

2.6. Le lien avec la feuille de route de Gefradis

Gefradis développe déjà un RAG pour la connaissance produit (accompagnement client). L'outil de bannières pourrait, en V2, se brancher sur cette même base. C'est cohérent avec leur volonté de multiplier les projets IA.

Phrase à dire à l'oral

« Pour le test, je n'ai pas mis de RAG : le cœur IA, c'est le workflow LLM (du brief vers le copy, puis vers la mise en page), et la charte tient dans le prompt. En V2, un RAG aurait une vraie valeur pour sélectionner automatiquement les visuels produit, réutiliser les campagnes passées et exploiter une charte détaillée qui évolue. C'est une question d'échelle : on l'ajoute quand la base documentaire devient assez grande pour le justifier. Il pourrait d'ailleurs se brancher sur le RAG produit que vous développez déjà. »

3. Comment l'application respecte toute la charte**3.1. Deux familles de règles, deux endroits**

La charte se divise en deux types de règles, respectées à deux endroits distincts. C'est le point central de l'architecture.

3.2. La charte visuelle : dans les gabarits et le code (déterministe)

Les règles visuelles sont **codées en dur** dans les gabarits HTML/CSS (dossier `app/templates/`) :

- **Couleurs exactes** (#253081, #FFD52B, #FFFDE5) déclarées en variables CSS.
- **Police Cabin, graisse 700 maximum** : on s'interdit d'aller au-delà, même si la police variable le permettrait.
- **Coins droits** : aucun arrondi.
- **Structure de marque** : carte bleue, titre blanc, chiffre dans le bloc jaune, figée dans le gabarit.
- **Dimensions exactes** : fixées par Python et Playwright pour chaque format.

3.3. La charte éditoriale : dans le prompt système

Ce que les gabarits ne peuvent pas garantir, c'est la **voix** du texte. Le prompt système porte donc le contexte de marque (charte 4 : fabricant français, Dunkerque depuis 1996, direct usine, sur-mesure) et le ton (charte 6 : professeur ou mentor, précis, transparent), pour que le copy sonne Gefradis.

3.4. La garantie architecturale

Claude génère **uniquement du JSON** (le texte et le choix de variante), **jamais le HTML/CSS**. Il remplit seulement les emplacements texte. Il lui est donc **physiquement impossible** d'introduire une couleur, une police ou une forme hors charte. La créativité de l'IA reste enfermée dans des briques déjà conformes.

3.5. En toute transparence : ce qui n'est pas encore en place

Le logo n'est pas encore intégré aux gabarits (en attente du fichier vectoriel) ; la palette complète (Midnight, Ghost white) et les photos produit ne sont pas encore utilisées (variante typographique d'abord).

Phrase à dire à l'oral

« La charte est respectée à deux niveaux : le visuel (couleurs, police, formes, dimensions) est codé en dur dans les gabarits, donc déterministe ; l'éditorial (le ton, le positionnement) est porté par le prompt système. Et comme Claude ne génère que du texte, jamais le code visuel, il lui est impossible de sortir de la charte graphique. »

4. Résumer la charte dans le prompt fait-il perdre du contexte ?

Résumer le contexte de marque pour le prompt système ne fait pas perdre d'information importante, pour quatre raisons.

- **Seule la charte éditoriale va dans le prompt.** La charte visuelle n'y va pas du tout (elle est dans les gabarits). Résumer le prompt ne perd donc aucune règle visuelle : Claude n'a pas besoin de connaître les codes couleur, il ne dessine pas.
- **Résumer n'est pas perdre.** On garde les consignes opérationnelles (descripteurs de ton exacts, faits de marque clés) et on enlève seulement le remplissage explicatif.
- **Un prompt court et ciblé est souvent meilleur qu'un prompt gonflé.** Trop de texte dilue l'attention du modèle et coûte plus cher. Le but n'est pas le maximum de contexte, mais le bon contexte.
- **La charte est petite.** Le contexte de marque et le ton tiennent en un ou deux paragraphes : le résumé est léger.

Garde-fous si le ton dérive : garder des consignes concrètes (à faire, à éviter) ; ajouter un ou deux exemples de bon copy dans le prompt (un exemple transmet le ton mieux qu'une description) ; itérer sur la sortie réelle et réinjecter la nuance manquante. C'est du prompt engineering normal.

Phrase à dire à l'oral

« Résumer la charte pour le prompt ne fait pas perdre de contexte : seule la partie éditoriale y va, la partie visuelle est dans le code. Et un prompt court et ciblé est souvent plus efficace qu'un prompt gonflé, qui dilue l'attention du modèle. Au besoin, j'ajoute des exemples et j'itère sur la sortie. »

5. Pourquoi Claude ne génère pas le HTML/CSS**5.1. Deux généralités à distinguer**

L'outil doit être générique, car il sert des **opérations commerciales** variées. Mais il faut distinguer :

- la généralité du **contenu** (n'importe quelle offre, n'importe quelle opération) : **nécessaire**, et c'est le rôle de Claude ;
- la liberté **visuelle** (mise en page et CSS au gré de l'IA) : **risquée**, et inutile.

Laisser Claude produire le HTML/CSS, c'est lui donner la liberté visuelle. On s'en abstient volontairement.

5.2. Les risques si l'IA génère le HTML/CSS

- **Fidélité à la charte : plus aucune garantie.** L'IA pourrait sortir une couleur hors palette, une autre police, une graisse interdite, des coins arrondis. Avec des gabarits fixes, c'est impossible.

- **Dimensions exactes : cassées.** Rien ne garantit qu'un HTML généré remplisse pile le format sans déborder.
- **Fiabilité.** Un JSON est validé contre un schéma (tool use) ; du HTML libre peut être tronqué (limite de tokens) ou mal formé, et le rendu casse.
- **Cohérence du kit.** Les formats d'une même campagne risqueraient d'être incohérents entre eux.
- **Assets.** Logo, images produit, police, accents : c'est le code qui gère ces chemins avec précision.
- **Coût et débogage.** Plus de tokens, et un HTML généré est très dur à tester et à maintenir.

5.3. La bonne façon d'être générique

Claude reste cantonné à un **JSON structuré** ; le code rend ce JSON dans des gabarits flexibles. Pour l'offre, Claude indique un **type** (montant, pourcentage ou texte) et le code choisit le rendu adapté, **toujours à la charte**. On obtient la genericité du *contenu* sans sacrifier la fidélité visuelle, les dimensions et la fiabilité.

Phrase à dire à l'oral

« Je contrains volontairement l'IA au texte structuré : c'est le code qui garantit la charte, les dimensions exactes et la cohérence du kit. Si je laissais l'IA générer le HTML, je perdrais ces garanties pour rien : la genericité dont j'ai besoin est dans le contenu, pas dans la liberté visuelle. »